Flutter: Cross-Platform Mobile Development

Long Questions · Short Questions · Code Practice · From Chapters 1–3 and Lecture 2

#	Long question	Source	Priority
1	What is Flutter? Why use it?	Chapter 1	Must do
2	Flutter Project Structure	Lecture 2 · Ch 1	Must do
3	Native vs Cross-platform Development	Chapter 3	Must do
4	Dart Variables, Data Types and Null Safety	Chapter 2	Must do
5	Dart Collections: List, Set, Map	Chapter 2	Must do
6	First Flutter App and Widget Tree	Chapter 1 · Lecture 2	Must do
7	Flutter vs React Native vs Xamarin vs Native	Chapter 3	Backup
8	Strengths, Weaknesses and Choosing a Framework	Chapter 3	Backup
9	Control Flow, Loops and Functions in Dart	Chapter 2	Backup
10	Hot Reload, Hot Restart and Build Modes	Chapter 1	Backup

Inside: Part A: 10 long questions · Part B: 33 short questions · Part C: code practice (homework programs and output questions).

How to write a long: Introduction → Table or points → Code or example → Conclusion.

PART A: LONG QUESTIONS

LONG 1: What is Flutter? Why use it?

Chapter 1 (sections 1–4)

Q. What is Flutter? Why do developers choose Flutter?

Introduction

Flutter is an **open-source UI framework (SDK)** for building apps. We write the code **once in Dart** and build apps for **Android, iOS** and other platforms. This is called **cross-platform development**.

How Flutter works

- **Dart** describes the app and its widgets.
- Flutter builds a **widget tree** (a hierarchy of widgets).
- Flutter **paints its own UI** using its rendering system, instead of only using Android/iOS controls.
- It talks to **platform services** when needed.

Why developers choose Flutter

Reason	Meaning
1. Shared development	Same UI and logic used for Android and iOS
2. Fast changes	Hot reload shows changes quickly
3. Consistent UI	App looks the same on all devices
4. Composability	Small widgets combine to make big screens
5. Open source	Free, with documentation, packages and community

Example: a university notice-board app shows the same notices and timetable on Android and iPhone from one codebase.

Balanced view: cross-platform does **not** mean zero platform work. Permissions, notifications and payments may need separate setup.

Conclusion: Flutter saves time because one Dart codebase can build apps for many platforms.

Yaad karne ki trick: S-F-C-C-O: "**S**ab **F**ast **C**hai **C**up **O**rder karo" (Shared, Fast, Consistent, Composable, Open source)

LONG 2: Flutter Project Structure

Lecture 2 · Chapter 1 (section 6)

Q. Explain the structure of a Flutter project.

Introduction

When we create a Flutter project, Flutter makes a **standard folder structure**. Each folder and file has a **specific purpose**.

```
my_first_app/
├── android/
├── ios/
├── lib/
│   └── main.dart
├── test/
├── web/  windows/  macos/  linux/
├── pubspec.yaml
├── pubspec.lock
├── analysis_options.yaml
├── README.md
└── .gitignore
```

Folder / File	Purpose
lib/	Main app code. **Most important folder**
lib/main.dart	**Entry point**; has main() and the root widget
android/	Android settings: permissions, app ID, Firebase, signing
ios/	iOS settings: permissions, bundle ID, signing (needs macOS + Xcode)
web/, windows/, macos/, linux/	Settings for web and desktop apps (web/index.html)
test/	Automated tests (widget_test.dart)
pubspec.yaml	Project name, **dependencies**, fonts and assets
pubspec.lock	**Exact versions** of installed packages (do not edit)
analysis_options.yaml	Linting rules: finds unused variables and style issues
.gitignore	Files Git should not upload (build/, .dart_tool/)
build/	Files made by **flutter build apk** (do not edit)

Execution flow



Conclusion: A beginner should focus mainly on **lib/main.dart** and **pubspec.yaml**.

Yaad karne ki trick: lib mein code · **pubspec** mein packages · **android/ios** mein settings · **test** mein testing

LONG 3: Native vs Cross-platform Development

Chapter 3 (sections 2–3)

Q. Differentiate native and cross-platform (Flutter) development.

Definitions

- **Native development:** making an app with the platform's **official language and tools**, e.g. **Kotlin/Java** for Android and **Swift** for iOS.
- **Cross-platform development: reusing code** across operating systems, e.g. Flutter uses one Dart codebase.

Aspect	Native approach	Flutter approach
Languages	Separate for each platform	Dart for shared code
UI	Platform-specific UI	Flutter widgets and rendering
Device features	Direct platform APIs	Plugins when needed
Maintenance	Separate projects	Many changes shared
Best for	Deep platform-specific apps	Same interface on all platforms

Example: a **campus café ordering app** has the same screens on Android and iOS, so Flutter fits. A special **AR app** that uses new device features may need native work.

Conclusion: Choose based on the **team, UI needs, device features and budget**. Neither approach wins every time.

Yaad karne ki trick: Native = **alag alag** app · Cross-platform = **ek code, kai platform**

LONG 4: Dart Variables, Data Types and Null Safety

Chapter 2 (sections 1–4)

Q. Explain variables, data types, final/const and null safety in Dart.

Introduction

Dart is the language used for Flutter. A **variable** stores a value under a name. Every Dart program starts from **main()**.

Type	Use	Example
int	Whole numbers	int age = 20;
double	Decimal numbers	double gpa = 3.4;
String	Text	String name = 'Ayesha';
bool	true / false	bool isEnrolled = true;
var	Dart guesses the type	var city = 'Faisalabad';

final, const and null safety

Keyword	Meaning
final	Value is set **once at runtime**
const	Value is fixed at **compile time**
String?	Nullable: the value **can be null**
??	Use a **fallback** if the left side is null
?.	Use a member **only if not null**

```
void main() {
  String name = 'Ayesha';
  int age = 20;
  final university = 'Campus A';
  const daysInWeek = 7;
  String? nickname;

  print('$name is $age years old.');   // Ayesha is 20 years old.
  print('Next year: ${age + 1}');      // Next year: 21
  print(nickname ?? 'No nickname');    // No nickname
  print(nickname?.length);             // null
}
```

Common mistake: `String name = null;` is an **error**. Use `String?` if the value can be missing.

Conclusion: Dart types and null safety make programs **safer** and reduce errors.

Yaad karne ki trick: final = run time par ek dafa · **const** = pehle se fix · **?** = null ho sakta hai · **??** = warna yeh

LONG 5: Dart Collections: List, Set, Map

Chapter 2 (section 5)

Q. Explain List, Set and Map in Dart with examples.

Introduction

Collections store **many values** in one variable.

Type	Example	Meaning
List	["Ali", "Sara"]	**Ordered**; index starts at **0**; duplicates allowed
Set	{"red", "blue"}	**Unique** values only; **no duplicates**
Map	{"name": "Ali"}	**Key-value** pairs

```
void main() {
  List<int> marks = [80, 75, 92];
  marks.add(88);

  Set<String> cities = {'Lahore', 'Lahore', 'Karachi'};
  Map<String, int> stock = {'earbuds': 12, 'speaker': 5};

  print(marks[0]);         // 80
  print(marks.length);     // 4
  print(cities.length);    // 2  (duplicate removed)
  print(stock['earbuds']); // 12
}
```

Conclusion: Use a **List** for ordered items, a **Set** for unique items and a **Map** for key-value data.

Yaad karne ki trick: List = line (order) · **Set** = sirf unique · **Map** = key se value

LONG 6: First Flutter App and Widget Tree

Chapter 1 (sections 5, 7) · Lecture 2

Q. Write a simple Flutter app and explain its parts and widget tree.

Introduction

A Flutter screen is built from **widgets**. Widgets are placed **inside** each other to make a **widget tree**.

```
import 'package:flutter/material.dart';

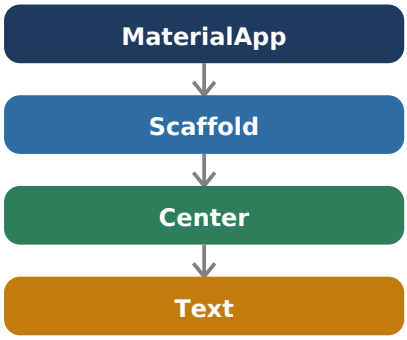
void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: Scaffold(
        body: Center(child: Text('Hello, Students!')),
      ),
    );
  }
}
```

Part	Job
main()	Program starts here
runApp()	Shows the root widget on screen
MaterialApp	App-level Material design settings
Scaffold	Basic screen structure (app bar, body)
Center	Puts the child in the middle
Text	Shows text

Widget tree



Commands to run the app

Command	Job
flutter doctor	Checks the setup
flutter create campus_notice	Makes a new project
flutter devices	Lists connected devices
flutter run	Builds and runs the debug app

Conclusion: Every Flutter screen is a **tree of widgets**, starting from main() and runApp().

Yaad karne ki trick: Widget tree: **M-S-C-T** → "**M**era **S**cool **C**ollege **T**ak" (MaterialApp, Scaffold, Center, Text)

In the paper: draw the widget tree as boxes with arrows. It is a homework question, so it may come.

LONG 7: Flutter vs React Native vs Xamarin vs Native

Chapter 3 (section 4)

Q. Compare Flutter with other mobile development approaches.

Approach	Main idea	Things to consider
Flutter	Dart framework that **draws its own** widget UI	Very consistent UI; must learn Dart; check plugins
React Native	Uses **JavaScript/TypeScript** and React	Good if the team knows React; check native modules
Xamarin	**C#/.NET** for mobile apps	Now **legacy**; use **.NET MAUI** for new projects
Native Android/iOS	Official languages and tools (Kotlin, Swift)	Best device access; separate codebases

Note: Xamarin is in the book for history. It is **not** the default choice for new apps today.

Conclusion: Each tool has trade-offs. Choose based on **team skills and app needs**.

Yaad karne ki trick: Flutter = **Dart** · React Native = **JavaScript** · Xamarin = **C#** (purana) · Native = **Kotlin/Swift**

LONG 8: Strengths, Weaknesses and Choosing a Framework

Chapter 3 (sections 5–9)

Q. Explain the strengths and weaknesses of Flutter. How do we choose a framework?

Strengths and weaknesses

Strength	Weakness / Question
Reuse of much code	Some platform work remains
Flexible UI with widgets	Must learn Dart and widgets
Fast changes with hot reload	Hot reload is for development only, not app speed
Rich animations	App size and performance must be tested
Many packages	Package quality can vary

How to choose a framework (criteria)

- **Developer skill:** which language does the team know?
- **Visual consistency:** must both apps look the same?
- **Device integrations:** camera, Bluetooth, special SDKs?
- **Library availability:** are good packages available?
- **Testing:** can we test on real devices?
- **Long-term support:** is the framework maintained?

Proof of Concept (POC): a **small test project** to check that an important requirement works before building the full app.

Example: a university information app (announcements, calendar, notifications) suits Flutter because most screens are shared.

Conclusion: Framework choice is a **trade-off** based on requirements, not a slogan.

Yaad karne ki trick: Criteria: "**D**il **V**ich **D**ard **L**ekin **T**ension **L**ow" (Developer, Visual, Device, Library, Testing, Long-term)

LONG 9: Control Flow, Loops and Functions in Dart

Chapter 2 (sections 6–10)

Q. Explain if/else, loops, switch and functions in Dart with examples.

if / else

```
int marks = 67;
if (marks >= 50) {
  print('Pass');        // Pass
} else {
  print('Try again');
}
```

Loops

Loop	When to use
for	When you know how many times
while	When it depends on a condition
for-in	To visit every item in a list

```
for (int i = 1; i <= 3; i++) {
  print('Lecture $i');   // Lecture 1, Lecture 2, Lecture 3
}
int count = 0;
while (count < 2) {
  print(count);          // 0, 1
  count++;
}
for (final item in ['Pen', 'Book']) {
  print(item);           // Pen, Book
}
```

switch

```
String grade = 'B';
switch (grade) {
  case 'A': print('Excellent'); break;
  case 'B': print('Good'); break;      // Good
  default: print('Keep practising');
}
```

Functions

A function is a **named set of instructions** that we can **reuse**. **Positional** arguments use their position; **named** arguments use labels; **required** makes a named argument compulsory.

```
int add(int first, int second) {
  return first + second;
}
String greet({required String name, String role = 'Student'}) {
  return 'Hello $name ($role)';
}
void main() {
  print(add(4, 6));            // 10
  print(greet(name: 'Sara'));  // Hello Sara (Student)
}
```

Conclusion: if/else makes **decisions**, loops **repeat** work, and functions **reuse** code.

Yaad karne ki trick: for = ginti pata · **while** = shart tak · **for-in** = list ka har item

LONG 10: Hot Reload, Hot Restart and Build Modes

Chapter 1 (sections 8–9)

Q. Explain hot reload, hot restart and the build modes of Flutter.

Hot reload vs Hot restart

Action	What happens
Hot reload	Applies code changes **quickly** and **keeps the app state**
Hot restart	Restarts the app and **resets the state**
Full restart	Rebuilds the app when **native or config** changes are made

Build modes

Mode	Use
Debug	While coding: debugging and hot reload
Profile	To **measure performance**
Release	**Optimized** app for end users

Example: change "Hello, Students!" to your department name and press hot reload: the text changes at once without losing state.

Conclusion: Hot reload makes development fast, and the release mode gives users a fast, optimized app.

Yaad karne ki trick: Build modes: **D-P-R** → **D**ebug (banate waqt), **P**rofile (speed check), **R**elease (users ke liye)

PART B: SHORT QUESTIONS

Q1. What is Flutter? (Ch 1)
An **open-source UI framework (SDK)** for building apps for many platforms from **one Dart codebase**.

Q2. What is Dart? (Ch 1)
The **programming language** used to write Flutter apps.

Q3. What is a widget? (Ch 1)
A **building block** of the screen, such as Text, Row, Button or Padding. Flutter screens are made of widgets.

Q4. What is an SDK? (Ch 1)
A collection of **tools, libraries and commands** for developers.

Q5. What is a codebase? (Ch 1)
The **source files** that define the application.

Q6. What is hot reload? (Ch 1)
It updates the running debug app after code changes while **keeping the state**.

Q7. Hot reload vs hot restart? (Ch 1)
Hot reload **keeps** the state; hot restart **resets** the state.

Q8. Name the build modes. (Ch 1)
Debug (coding), **Profile** (performance), **Release** (end users).

Q9. What does flutter doctor do? (Ch 1)
It **checks the setup** and reports problems. (flutter create makes a project, flutter run runs it.)

Q10. Where are dependencies and assets declared? (Ch 1)
In **pubspec.yaml**.

Q11. Why do we use a device or emulator? (Ch 1)
To **run, observe and test** the app.

Q12. Does shared code remove all native configuration? (Ch 1)
No. Permissions, notifications and some plugins still need platform settings.

Q13. What is main.dart? (Lec 2)
The **entry point** of the app. It has **main()** and the root widget.

Q14. Write the execution flow of a Flutter app. (Lec 2)
main() → runApp() → MyApp → build() → widgets on screen

Q15. pubspec.yaml vs pubspec.lock? (Lec 2)
pubspec.yaml lists dependencies; **pubspec.lock** records their **exact installed versions**.

Q16. What does flutter pub get do? (Lec 2)
It **downloads** the packages listed in pubspec.yaml.

Q17. What is analysis_options.yaml? (Lec 2)
It sets **linting rules** that find unused variables, bugs and style issues.

Q18. What is .gitignore? (Lec 2)
It tells Git which files **not to upload**, e.g. build/ and .dart_tool/.

Q19. What is the test/ folder? (Lec 2)
It contains **automated tests**, e.g. widget_test.dart.

Q20. What does main() do? (Ch 2)
It is the **starting point** of a Dart program.

Q21. final vs const? (Ch 2)
final is assigned once at **runtime**; **const** is a **compile-time** constant.

Q22. String vs String? (Ch 2)
String? can be **null**; **String** cannot.

Q23. What do ?? and ?. do? (Ch 2)
?? gives a fallback value if null; **?.** uses a member only if the value is not null.

Q24. What is string interpolation? (Ch 2)
Putting values inside a string: **$name** for a variable, **${age + 1}** for an expression.

Q25. List vs Set vs Map? (Ch 2)
List = **ordered**; Set = **unique** values; Map = **key-value** pairs.

Q26. Why use a Map? (Ch 2)
To store values using **keys**, e.g. product name and stock count.

Q27. Positional vs named arguments? (Ch 2)
Positional use **position**: add(4, 6). Named use **labels**: greet(name: "Sara"). **required** makes a named one compulsory.

Q28. Why create functions? (Ch 2)
To **organize** code and **reuse** logic.

Q29. What is native development? (Ch 3)
Building an app with a platform's **own tools and APIs**, e.g. Kotlin for Android, Swift for iOS.

Q30. Does cross-platform mean every line is reusable? (Ch 3)
No. Some platform-specific work is still needed.

Q31. What is a Proof of Concept (POC)? (Ch 3)
A **small test project** to check that an important requirement is possible.

Q32. Is Xamarin recommended for new apps today? (Ch 3)
No. It is legacy; newer **.NET MAUI** should be checked.

Q33. Why check packages before using them? (Ch 3)
Package **quality, maintenance and licence** vary, like checking ingredients before cooking.

PART C: CODE PRACTICE

These are the **homework programs** from the chapters, solved simply. Outputs are shown as comments.

1. Print student details (variables)

```
void main() {
  String name = 'Ali';
  int semester = 5;
  bool feePaid = true;
  print('$name is in semester $semester. Fee paid: $feePaid');
}
// Output: Ali is in semester 5. Fee paid: true
```

2. Total of five marks using a loop

```
void main() {
  List<int> marks = [70, 80, 65, 90, 75];
  int total = 0;
  for (final m in marks) {
    total += m;
  }
  print('Total: $total');
}
// Output: Total: 380
```

3. Function for discounted price

```
double discountedPrice(double price, double discount) {
  return price - (price * discount / 100);
}
void main() {
  print(discountedPrice(1000, 10));
}
// Output: 900.0
```

4. Map of subjects and grades

```
void main() {
  Map<String, String> grades = {'Maths': 'A', 'Physics': 'B', 'English': 'A'};
  print(grades['Physics']);
}
// Output: B
```

5. Grade calculator with a function (classroom activity)

```
String getGrade(int score) {
  if (score >= 80) {
    return 'A';
  } else if (score >= 65) {
    return 'B';
  } else if (score >= 50) {
    return 'C';
  } else {
    return 'F';
  }
}
void main() {
  print(getGrade(72));
}
// Output: B
```

6. Class average (from the chapter)

```
double average(List<int> marks) {
  int total = 0;
  for (final mark in marks) {
    total += mark;
  }
  return total / marks.length;
}
void main() {
  print(average([60, 70, 80]));
}
// Output: 70.0
```

7. Output questions (what will be printed?)

Code	Output
print(2 + 3);	5
Set: {'Lahore', 'Lahore', 'Karachi'}.length	2
List [80, 75, 92] after add(88): marks[0]	80
List [80, 75, 92] after add(88): marks.length	4
String? nickname; print(nickname ?? 'No nickname');	No nickname
String? nickname; print(nickname?.length);	null
int age = 20; print('${age + 1}');	21
switch on 'B' (A: Excellent, B: Good)	Good
while (count < 2) starting at 0	0 then 1
greet(name: 'Sara')	Hello Sara (Student)
String name = null;	**Error** (use String?)

Tip for code questions: write main(), end every statement with a semicolon (;), and write the output as a comment.